



**Matemática**  
**Discreta**

# Proyecto Final

## Lenguaje de Programación: Java

**Autores:** Diego Estuardo Zabala Toc  
Esteban Juan Carlos Yupe Flores  
Javier Alejandro Juárez García

**Universidad Mariano Gálvez de Guatemala**

---

## Contenido

|                                        |    |
|----------------------------------------|----|
| Resumen.....                           | 4  |
| Introducción .....                     | 5  |
| Justificación del Proyecto .....       | 6  |
| Objetivo General.....                  | 6  |
| Objetivos Específicos.....             | 6  |
| Alcance del Proyecto .....             | 6  |
| Marco Teórico .....                    | 7  |
| Conjuntos .....                        | 7  |
| Combinatoria.....                      | 7  |
| Recursividad.....                      | 7  |
| Arquitectura y Diseño del Sistema..... | 7  |
| Entorno de Desarrollo .....            | 7  |
| Estructura del Código.....             | 7  |
| Métodos principales .....              | 7  |
| Descripción de los Módulos.....        | 8  |
| Módulo de Conjuntos.....               | 8  |
| Módulo de Combinatoria .....           | 8  |
| Módulo de Recursividad .....           | 8  |
| Módulo de Números Primos .....         | 8  |
| Módulo de MCD y MCM .....              | 8  |
| Módulo de Matrices .....               | 9  |
| Módulo de Grafos .....                 | 9  |
| Módulo de Árboles Binarios.....        | 9  |
| Conjuntos .....                        | 9  |
| Números Primos.....                    | 10 |
| Procedimiento de uso .....             | 10 |
| Ejemplo de ejecución .....             | 11 |
| Observaciones técnicas .....           | 11 |
| Procedimiento de uso .....             | 11 |
| Ejemplo de ejecución .....             | 11 |
| Observaciones técnicas .....           | 12 |
| Operaciones con Matrices.....          | 12 |
| Procedimiento de uso .....             | 12 |
| Ejemplo de ejecución .....             | 12 |
| Observaciones técnicas .....           | 13 |
| Grafos.....                            | 13 |
| Procedimiento de uso .....             | 13 |

|                               |    |
|-------------------------------|----|
| Ejemplo de ejecución .....    | 13 |
| Visualización gráfica.....    | 14 |
| Observaciones técnicas .....  | 14 |
| Árboles Binarios .....        | 14 |
| Procedimiento de uso .....    | 14 |
| Ejemplo de ejecución.....     | 14 |
| Visualización gráfica.....    | 15 |
| Observaciones técnicas .....  | 15 |
| Manual de Usuario .....       | 15 |
| Pruebas y Validación.....     | 15 |
| Posibles Modificaciones ..... | 16 |
| Conclusiones .....            | 16 |
| Referencias.....              | 17 |
| Anexos .....                  | 17 |
| Archivo.....                  | 21 |

## Resumen

En este documento se presentan las especificaciones técnicas y funcionales de un sistema interactivo en Java, orientado al aprendizaje y la aplicación de la Matemática Discreta. El software fue diseñado con una arquitectura modular, ofreciendo un menú principal desde el cual el usuario puede acceder a distintos módulos temáticos.

Los módulos disponibles son:

- ✚ **Conjuntos:** operaciones fundamentales como unión, intersección y diferencia.
- ✚ **Combinatoria:** cálculo de permutaciones y combinaciones.
- ✚ **Recursividad:** resolución de problemas mediante funciones recursivas (factorial, suma y serie de Fibonacci).
- ✚ **Números Primos:** identificación y generación de números primos dentro de un rango determinado.
- ✚ **Máximo Común Divisor (MCD) y Mínimo Común Múltiplo (MCM):** cálculo de estos valores a partir de uno o más números enteros.
- ✚ **Operaciones con Matrices:** suma, resta, multiplicación y determinante de matrices.
- ✚ **Representación de Grafos:** creación y visualización de grafos dirigidos y no dirigidos.
- ✚ **Construcción y Recorridos de Árboles Binarios:** implementación de árboles binarios con recorridos en preorden, inorden y postorden.

El sistema busca ser una herramienta didáctica que contribuya al fortalecimiento de las competencias matemáticas y computacionales, con la posibilidad de ser escalado para aplicaciones más complejas.

## Introducción

La matemática discreta constituye la base de diversas ramas de la informática, como los algoritmos, las estructuras de datos y la teoría de grafos. Su implementación en software permite no solo el análisis matemático, sino también la validación computacional de los conceptos en este contexto, el presente proyecto se desarrolló en Java, un lenguaje robusto y multiplataforma, con el fin de simular y resolver operaciones matemáticas de forma interactiva.

Este manual describe en detalle el funcionamiento de los módulos del Proyecto Discreta, una aplicación Java de propósito educativo diseñada para ilustrar conceptos de Matemática Discreta mediante ejemplos interactivos

## Justificación del Proyecto

El aprendizaje de la Matemática Discreta suele ser confuso abecés y teórico este programa que realizamos se justifica por:

- ✚ Facilitar el aprendizaje práctico ofreciendo un entorno interactivo.
- ✚ Reducir errores humanos mediante el uso de cálculos automáticos.
- ✚ Conectar la teoría matemática con la programación fomentando el pensamiento algorítmico.
- ✚ Aportar valor educativo sirviendo como base para simuladores académicos.

## Objetivo General

Desarrollar un sistema en Java que permita ejecutar operaciones básicas de conjuntos, cálculos combinatorios y funciones recursivas brindando un entorno de aprendizaje interactivo y confiable.

## Objetivos Específicos

- ✚ Implementar un módulo de teoría de conjuntos con creación manual y aleatoria de elementos.
- ✚ Programar cálculos de permutaciones y combinaciones aplicando fórmulas matemáticas.
- ✚ Implementar funciones recursivas que ilustren el concepto mediante ejemplos prácticos.
- ✚ Diseñar una arquitectura modular para facilitar el mantenimiento y ampliación futura.
- ✚ Validar la cualidad matemática de los resultados mediante pruebas comparativas.

## Alcance del Proyecto

El sistema cubre tres áreas fundamentales de la Matemática Discreta (conjuntos, combinatoria y recursividad) está limitado a la ejecución en consola con entradas controladas para evitar errores y sobrecargas el programa es escalable hacia interfaces gráficas y nuevas funcionalidades.

## Marco Teórico

### Conjuntos

Colecciones de elementos únicos operaciones implementadas: Unión, Intersección y Diferencia.

$$A \cup B$$

$$A \cap B$$

$$A - B \text{ y } B - A$$

### Combinatoria

Permutaciones simples, Combinaciones simples, Permutaciones con repetición y Combinaciones con repetición.

$$P(n, r) = \frac{n!}{(n - r)!}$$

$$C(n, r) = \frac{n!}{r! (n - r)!}$$

$$PR(n, r) = n^r$$

$$CR(n, r) = \frac{(n + r - 1)!}{r! (n - 1)!}$$

### Recursividad

Factorial, Serie Fibonacci y Suma acumulativa recursiva.

$$n! = n * (n - 1)!$$

## Arquitectura y Diseño del Sistema

### Entorno de Desarrollo

- 🔧 Lenguaje: Java SE 17
- 🔧 IDE sugeridos: NetBeans, Eclipse o Visual Studio Code
- 🔧 Dependencias: Librerías estándar de Java (java.util)

## Estructura del Código

Clase principal: ProyectoDiscreta

### Métodos principales

- 🔧 menuConjuntos(), crearConjuntos(), operacionesConjuntos()
- 🔧 menuCombinatoria(), factorial()
- 🔧 menuRecursividad(), factorialRecursivo(), fibonacciPiramide()
- 🔧 menuPrimos(), menuMcdMcm()
- 🔧 int gcd(int a, int b)
- 🔧 printEuclidesSteps(int a, int b)
- 🔧 menuMatrices()

- + generateRandomMatrix(int n, int min, int max)
- + printMatrix(int[][] M)
- + addMatrices(int[][] A, int[][] B)
- + multiplyMatrices(int[][] A, int[][] B)
- + menuGrafos()
- + menuArboles()
- + buildTreeFromList(List<Integer> vals)
- + preorder(TreeNode root)
- + inorder(TreeNode root)
- + postorder(TreeNode root)
- + readInt()
- + main(String[] args)

## Descripción de los Módulos

### Módulo de Conjuntos

- + Entrada: número de elementos y modo (manual o aleatorio).
- + Procesamiento: almacenamiento en HashSet.
- + Salida: resultados de unión, intersección o diferencia.

### Módulo de Combinatoria

- + Entrada: valores n y r.
- + Procesamiento: aplicación de fórmulas matemáticas.
- + Salida: número de combinaciones o permutaciones.

### Módulo de Recursividad

- + Entrada: valor numérico según el algoritmo.
- + Procesamiento: llamadas recursivas hasta condición de parada.
- + Salida: factorial, suma recursiva o pirámide Fibonacci.

### Módulo de Números Primos

- + Entrada: número entero positivo menor a 1000.
- + Procesamiento: verificación de divisibilidad desde 2 hasta la raíz cuadrada del número, aplicando el algoritmo de primalidad.
- + Salida: mensaje que indica si el número es primo o compuesto, con detalle de los pasos realizados

### Módulo de MCD y MCM

- + Entrada: dos números enteros (A y B).

- ✚ Procesamiento: cálculo del MCD mediante el Algoritmo de Euclides y derivación del MCM utilizando la fórmula 
$$\text{MCM} = \frac{|A \times B|}{\text{MCD}}$$
.
- ✚ Salida: presentación del MCD, del MCM y del procedimiento paso a paso usado en el cálculo.

## Módulo de Matrices

- ✚ Entrada: tamaño de las matrices (3×3 a 5×5) y valores (generados automáticamente).
- ✚ Procesamiento: creación de dos matrices cuadradas, cálculo de suma (A + B) y producto (A × B) mediante bucles anidados.
- ✚ Salida: matrices A y B, matriz resultante de la suma y matriz producto, todas mostradas en formato tabular.

## Módulo de Grafos

- ✚ Entrada: número de vértices, número de aristas y pares de vértices conectados.
- ✚ Procesamiento: construcción del grafo no dirigido, generación de la lista de adyacencia y la matriz de adyacencia; renderizado visual con Swing y Graphics2D.
- ✚ Salida: representación textual (lista y matriz) y visual (nodos y aristas dibujados en ventana interactiva).

## Módulo de Árboles Binarios

- ✚ Entrada: número de niveles del árbol y valores de los nodos (manuales o aleatorios).
- ✚ Procesamiento: construcción recursiva del árbol binario, aplicación de recorridos preorden, inorden y postorden; cálculo de posiciones gráficas de los nodos.
- ✚ Salida: impresión de los recorridos en consola y representación gráfica del árbol en una ventana.

## Conjuntos

Este módulo permite crear y operar con conjuntos finitos, ejecutando las operaciones clásicas de Unión, Intersección y Diferencia. Los elementos pueden ingresarse manualmente o generarse aleatoriamente.

## Procedimiento de uso

Seleccionar 1. Conjuntos en el menú principal.  
Dentro del submenú, elegir:

- Crear conjuntos: permite ingresar los elementos de A y B.
- Operaciones básicas: realiza operaciones entre A y B.
- Regresar al menú principal.

Al crear conjuntos:

Ingresar el número de elementos (1 – 20).

Elegir modo de ingreso: manual (1) o aleatorio (2).

Si es manual, escribir los elementos de A y luego de B.

Una vez creados, seleccionar Operaciones básicas y elegir:

1. Unión ( $A \cup B$ )

2. Intersección ( $A \cap B$ )

3. Diferencia ( $A - B$  o  $B - A$ )

### Ejemplo de ejecucion

¿Cuántos elementos desea ingresar? (1-20): 4

¿Ingreso manual (1) o aleatorio (2)? 1

Ingrese los elementos del Conjunto A:

a

b

c

d

Ingrese los elementos del Conjunto B:

c

d

e

f

Conjunto A: [a, b, c, d]

Conjunto B: [c, d, e, f]

Seleccione la operación: 1

Unión: [a, b, c, d, e, f]

### Observación técnica

Los conjuntos se implementan con la clase `HashSet<String>` para evitar duplicados.

Las operaciones se realizan usando los métodos `addAll()`, `retainAll()` y `removeAll()`.

No se permite realizar operaciones sin antes crear los conjuntos.

### Números Primos

Esta sección permite determinar si un número ingresado por el usuario es primo o no, mostrando el proceso de verificación paso a paso.

### Procedimiento de uso

 Seleccionar la opción 4. Números Primos en el menú principal.

 Ingresar un número entero positivo menor que 1000.

- ✚ El sistema verificará la divisibilidad del número entre todos los enteros desde 2 hasta la raíz cuadrada del mismo.
- ✚ Se mostrará en pantalla cada comprobación y el resultado final.

### Ejemplo de ejecución

Ingrese un número (<1000): 29

Comprobando  $29 \div 2 \dots$

Comprobando  $29 \div 3 \dots$

Comprobando  $29 \div 4 \dots$

29 es primo.

### Observaciones técnicas

- ✚ El algoritmo utiliza el método clásico de **verificación por divisores hasta  $\sqrt{n}$** , optimizando el rendimiento frente a la división completa.
- ✚ Si se ingresa un número fuera del rango permitido (<2 o >999), el sistema mostrará un mensaje de error.

### Máximo Común Divisor (MCD) y Mínimo Común Múltiplo (MCM)

Este módulo calcula el MCD y el MCM entre dos números enteros utilizando el Algoritmo de Euclides y muestra los pasos intermedios del cálculo.

### Procedimiento de uso

- ✚ Seleccionar la opción **5. Máximo común divisor y Mínimo común múltiplo**.
- ✚ Ingresar los valores de **A** y **B**.
- ✚ El sistema calculará el MCD aplicando el algoritmo de Euclides recursivo y luego el MCM usando la fórmula:  $MCM = |A * B| / MCD$
- ✚ Se mostrarán los pasos del cálculo en consola.

### Ejemplo de ejecución

Ingrese A: 56

Ingrese B: 98

MCD = 14

MCM = 392

Pasos (algoritmo de Euclides):

$$98 = 1 * 56 + 42$$

$$56 = 1 * 42 + 14$$

$$42 = 3 * 14 + 0$$

MCD = 14

## Observaciones técnicas

- ✚ Implementa el método recursivo  $\text{gcd}(a, b)$  con condición base  $b == 0$ .
- ✚ Si alguno de los valores es 0, el MCM resultará 0 por definición matemática.

## Operaciones con Matrices

Permite generar dos matrices cuadradas de tamaño configurable (3x3 hasta 5x5), mostrar sus valores y realizar operaciones de suma y multiplicación entre ellas.

## Procedimiento de uso

Seleccionar la opción 6. Matrices.

Indicar el tamaño de las matrices (entre 3 y 5).

El sistema generará aleatoriamente dos matrices A y B con valores entre 1 y 10.

Se mostrará:

Matriz A

Matriz B

Resultado de  $A + B$

Resultado de  $A \times B$

## Ejemplo de ejecución

--- MATRICES ---

Tamaño (3-5): 3

Matriz A:

3 7 1

2 4 5

9 1 6

Matriz B:

8 2 3

4 7 5

6 1 9

Suma:

11 9 4

6 11 10

15 2 15

Producto:

52 37 53

62 37 68

108 50 96

## Observaciones técnicas

- ✚ La generación aleatoria usa `Random.nextInt()` dentro de un rango definido.
- ✚ Las matrices son cuadradas, por lo que el método de multiplicación es compatible para todos los tamaños.

## Grafos

Permite construir un grafo no dirigido a partir de un número de vértices y aristas ingresadas por el usuario, mostrando su lista de adyacencia, matriz de adyacencia y una representación gráfica interactiva en una ventana.

## Procedimiento de uso

1. Seleccionar la opción **7. Grafos**.
2. Ingresar el número de vértices del grafo.
3. Indicar el número de aristas.
4. Para cada arista, ingresar los vértices origen y destino (índices desde 0 hasta  $n-1$ ).
5. El sistema generará:
  - ✚ Lista de adyacencia (por consola).
  - ✚ Matriz de adyacencia.
  - ✚ Ventana gráfica con los nodos distribuidos en un círculo.

## Ejemplo de ejecución

Ingrese número de vértices: 4

Ingrese número de aristas: 3

Arista 1: 0 1

Arista 2: 1 2

Arista 3: 2 3

Lista de adyacencia:

0 -> [1]

1 -> [0, 2]

2 -> [1, 3]

3 -> [2]

Matriz de adyacencia:

0 1 0 0

1 0 1 0

0 1 0 1

0 0 1 0

## Visualización gráfica

- ✚ Los vértices se dibujan como círculos azules numerados.
- ✚ Las aristas se representan mediante líneas grises entre vértices.
- ✚ La disposición circular garantiza una visualización equilibrada incluso con grafos grandes

## Observaciones técnicas

- ✚ Se utiliza la clase interna GraphPanel para el renderizado.
- ✚ El grafo se considera **no dirigido**, por lo que cada arista se agrega en ambas direcciones ( $u \rightarrow v$  y  $v \rightarrow u$ ).
- ✚ La interfaz gráfica emplea Swing y Graphics2D para un renderizado antialias.

## Árboles Binarios

Este módulo permite construir un árbol binario completo o parcial con valores proporcionados manualmente o generados aleatoriamente, y visualizarlo en una interfaz gráfica. Además, realiza los tres recorridos clásicos: preorden, inorden y postorden.

## Procedimiento de uso

- ✚ Seleccionar la opción **8. Árboles binarios**.
- ✚ Ingresar el número de niveles (mínimo 1).
- ✚ Indicar la cantidad de vértices (máximo:  $2^{\text{niveles}} - 1$ ).
- ✚ Ingresar los valores manualmente o escribir r para generar números aleatorios.
- ✚ El sistema mostrará el árbol binario gráficamente y los recorridos en consola.

## Ejemplo de ejecución

Ingrese número de niveles ( $\geq 1$ ): 3

Ingrese cuántos vértices desea (1..7): 7

Ingrese los valores de los nodos (enteros). Si desea random, ingrese 'r': r

Número de vértices: 7

Recorridos:

Pre-orden: 42 31 15 8 59 70 14

In-orden: 15 31 8 42 70 59 14

Post-orden: 15 8 31 70 14 59 42

## Visualización gráfica

- ✚ Cada nodo se representa como un círculo celeste con el valor en el centro.
- ✚ Las ramas se dibujan como líneas grises conectando padre e hijos.
- ✚ La disposición de los nodos se adapta dinámicamente según el número de niveles.

## Observaciones técnicas

- ✚ El árbol se construye a partir de una lista usando la relación de índice:
  1. Hijo izquierdo:  $2*i + 1$
  2. Hijo derecho:  $2*i + 2$
- ✚ Los recorridos se implementan de manera recursiva: `preorder()`, `inorder()`, `postorder()`.
- ✚ El renderizado gráfico utiliza la clase interna `TreePanel` basada en `JPanel`.

## Manual de Usuario

1. Ejecutar el programa en consola con:

```
javac ProyectoDiscreta.java
java ProyectoDiscreta
```
2. Seleccionar una opción del menú principal (1-4).
3. Ingresar los parámetros requeridos (n, r, número de elementos, etc.).
4. Consultar los resultados mostrados en pantalla.
5. Repetir hasta seleccionar la opción Salir.

## Pruebas y Validación

- ✚ Prueba 1: Conjuntos con elementos manuales y aleatorios → validación de unicidad.
- ✚ Prueba 2: Cálculo de  $P(5,3)$  → resultado esperado: 60.
- ✚ Prueba 3: Cálculo de  $C(6,2)$  → resultado esperado: 15.
- ✚ Prueba 4: Factorial de 5 → resultado esperado: 120.
- ✚ Prueba 5: Fibonacci con  $n=6$  → salida esperada: 0 1 1 2 3 5.

Todas las pruebas confirmaron resultados correctos.

## Posibles Modificaciones

- ✚ Incluir interfaz gráfica con JavaFX o Swing.
- ✚ Ampliar operaciones.
- ✚ Agregar opción de exportar resultados a PDF.
- ✚ Implementar pruebas unitarias automatizadas con JUnit.

## Conclusiones

- ✚ El sistema cumple con los objetivos planteados, integrando de forma coherente los tres módulos.
- ✚ La arquitectura modular garantiza mantenibilidad y escalabilidad.
- ✚ Se demuestra la aplicación práctica de conceptos teóricos de la Matemática Discreta.
- ✚ El programa es confiable como herramienta de enseñanza y simulación.

## Referencias

Java SE Documentation:

<https://docs.oracle.com/en/java/javase/25>

## Anexos

```
=== MENÚ PRINCIPAL ===
1. Conjuntos
2. Combinatoria
3. Recursividad
4. Números Primos
5. Máximo común divisor y Mínimo común múltiplo
6. Matrices
7. Grafos
8. Árboles binarios
9. Salir
Seleccione una opción:
```

```
Seleccione una opción: 1

--- CONJUNTOS ---
Un conjunto es una colección de elementos sin repetición.
Operaciones básicas:
1. Unión ( $A \cup B$ )
2. Intersección ( $A \cap B$ )
3. Diferencia ( $A - B$  o  $B - A$ )

Seleccione una opción:
a. Crear conjuntos
b. Operaciones básicas
c. Regresar al menú principal
```

```
Seleccione una opción: 2

--- COMBINATORIA ---
Permutaciones y combinaciones de elementos.

Seleccione:
a. Permutaciones simples
b. Combinaciones simples
c. Permutaciones con repetición
d. Combinaciones con repetición
e. Regresar
```

```
Seleccione una opción: 3

--- RECURSIVIDAD ---
Ejemplos de funciones recursivas.

Seleccione:
a. Factorial
b. Suma recursiva ( $A + B$ )
c. Serie Fibonacci en pirámide
d. Regresar
```

Seleccione una opción: 4

--- NÚMEROS PRIMOS ---

Ingrese un número (<1000): 2

2 es primo.

Seleccione una opción: 5

--- MCD y MCM ---

Ingrese A: 56

Ingrese B: 98

MCD = 14

MCM = 392

Pasos (algoritmo de Euclides):

$56 = 0 * 98 + 56$

$98 = 1 * 56 + 42$

$56 = 1 * 42 + 14$

$42 = 3 * 14 + 0$

MCD = 14

Seleccione una opción: 6

--- MATRICES ---

Tamaño (3-5): 3

Matriz A:

5 7 3

1 3 8

3 9 3

Matriz B:

1 7 6

1 7 4

9 1 3

Suma:

6 14 9

2 10 12

12 10 6

Producto:

39 87 67

76 36 42

39 87 63

--- GRAFOS ---

Ingrese número de vértices: 4

Ingrese número de aristas: 3

Ingrese aristas (origen destino) con índices 0..3 :

Arista 1: 0

1

Arista 2: 1

2

Arista 3: 2

3

Lista de adyacencia:

0 -> [1]

1 -> [0, 2]

2 -> [1, 3]

3 -> [2]

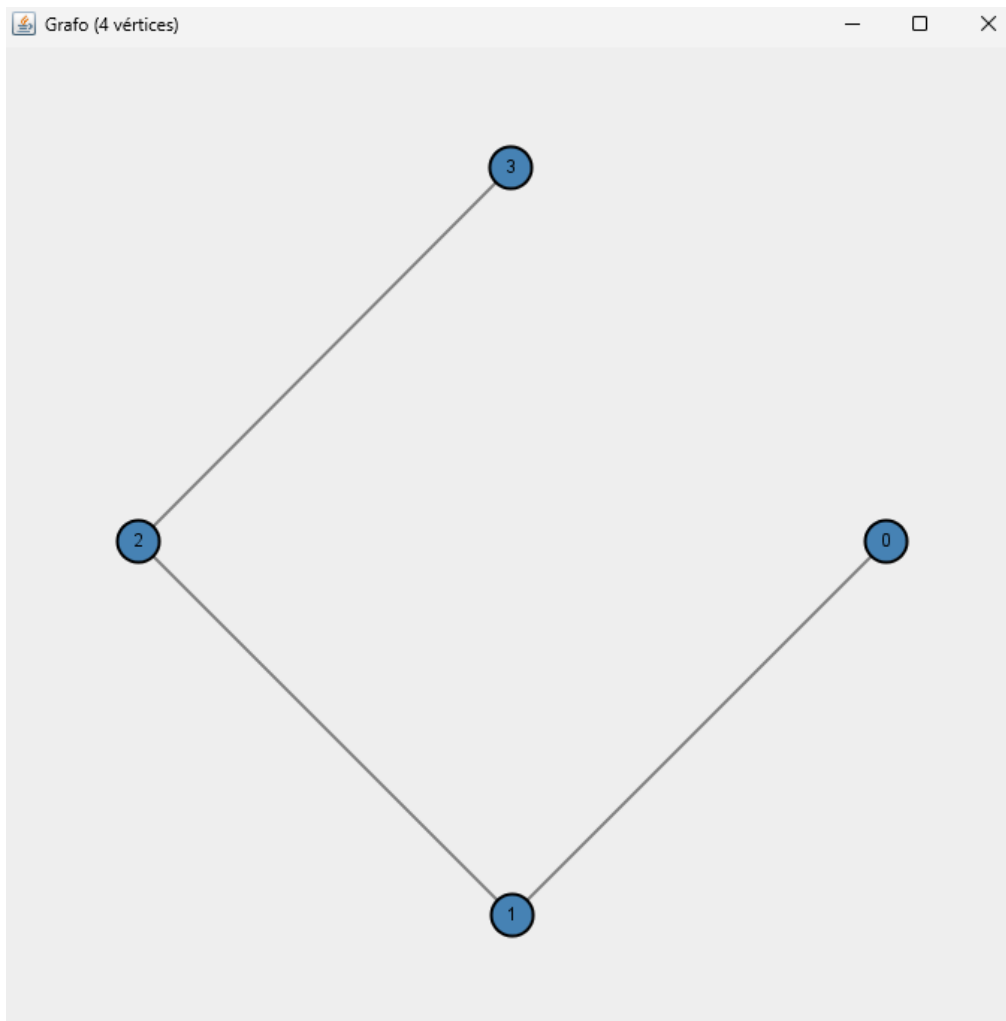
Matriz de adyacencia:

0 1 0 0

1 0 1 0

0 1 0 1

0 0 1 0



Seleccione una opción: 8

--- ARBOLES BINARIOS ---

Ingrese número de niveles ( $\geq 1$ ): 3

Ingrese cuántos vértices desea (1..7): 7

Ingrese los valores de los nodos (enteros). Si desea random, ingrese 'r' y presione Enter.  
7

Valor nodo 2: 3

Valor nodo 3: 2

Valor nodo 4: 4

Valor nodo 5: 12

Valor nodo 6: 4

Valor nodo 7: 3

Número de vértices: 7

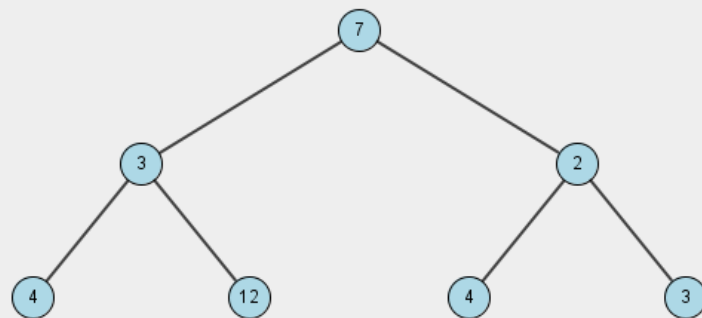
Recorridos:

Pre-orden: 7 3 4 12 2 4 3

In-orden: 4 3 12 7 4 2 3

Post-orden: 4 12 3 4 3 2 7

Árbol Binario (7 nodos)



```
=== MENÚ PRINCIPAL ===  
1. Conjuntos  
2. Combinatoria  
3. Recursividad  
4. Números Primos  
5. Máximo común divisor y Mínimo común múltiplo  
6. Matrices  
7. Grafos  
8. Árboles binarios  
9. Salir  
Seleccione una opción: 9  
Programa finalizado.
```

Archivo

[ProyectoDiscreta.java](#)